

EZ

- T1 消消乐
- T2 打乱数组
- T3 两数之差
- T4 01串
- T5 接雨水
- T6 只删一个
- T7 最大子段和
- T8 区间和恰好为 k
- T9 最长不相交区间
- T10 安排房间
- T11 铺设道路
- T12 绝对值之和
- T13 区间异或和

MID

- T1 分数规划 I
- T2 分数规划 II
- T3 分数规划 III
- T4 棱形前缀和
- T5 摧毁路线
- T6 代价转移
- T7 最大正方形
- T8 最大棱形
- T9 位运算
- T10 中位数

HARD

- T1 区间 LCA

UNK

收集一些应当必备（大概）的 trick，由于本人是 ACMer，所以会更加面向 ACM 一点。

保证 EZ 难度均小于洛谷黄题，并且是萌新必练；MID 难度可能比较宽泛；HARD 由于本人太菜，所以比较少；UNK 是还没整理的 trick。

trick 来源于 Q群(835014459) 的群友们和本人平时的练习。

课后习题链接：<https://www.luogu.com.cn/training/844590>

希望能帮到您！

T1 消消乐

给定一个长度为 n 的数组 a ，每次可以选择两个不同下标 i, j ($i \neq j$ 且 $a_i \neq 0, a_j \neq 0$)，使 a_i, a_j 各减去 1，请最小化最终数组的和。 [\(链接\)](#)

推荐时间复杂度 $O(n)$ 。

tag

贪心

solve

我们先遍历数组，得到数组中的最大值 mx 和整个数组的和 sum 。如果 $mx \geq sum - mx$ ，那么结果就是 $2 \cdot mx - sum$ ；否则，结果就是 $sum \pmod 2$

证明：（提供者 xixii_）

数组 b ： a 中各种相同数字个数的集合
 $n = a.size$ ， $maxi = \text{ranges::max}(b)$
 ① n 偶，
 I. $maxi \leq \frac{n}{2}$ 时，其它元素可把它消完，再取剩余的 $maxi$ ，
 重复该过程， $ans = 0$ ($ans = 2 * maxi - n$)
 II. $maxi > \frac{n}{2}$ 时，用 $maxi$ 消其它元素， $ans = n - 2 * (n - maxi)$
 ② n 奇，显然多一个元素剩余，不妨设为 $maxi$ 中的元素
 考虑剩下 $n-1$ 个元素，回归①的计算，最后 $ans + 1$ 即可

洛谷

T2 打乱数组

给定两个长度为 n 的升序数组 a, b ，可以打乱 a 数组，记打乱后的 a 数组为 c 。请最小化/最大化 $\sum_{i=1}^n b_i \cdot c_i$ 的值。（没有链接）

推荐时间复杂度 $O(n \log n)$ 。

tag

贪心，数学

solve

对于两个升序数组，可以证明有：

$$a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \geq a_{p_1} b_1 + a_{p_2} b_2 + \cdots + a_{p_n} b_n \geq a_n b_1 + a_{n-1} b_2 + \cdots + a_1 b_n$$

其中 p 是任意长度为 n 的排列。这就是著名的[排序不等式](#)。

T3 两数之差

给定一个长度为 n 的数组 a ，请问数组中有多少数对 (i, j) 满足 $a_i - a_j = k$? [\(链接\)](#)

推荐时间复杂度 $O(n \log n)$ 。

tag

二分查找

solve

由于原式看起来不太好做，考虑将原式进行变形，得到： $a_j = a_i - k$ 。也就是说，对于每一个 a_i ，我们只需要看看数组中有几个 $a_j = a_i - k$ 就好了。

于是原问题转换为怎么查询一个数组中有多少个值等于 x 。这个问题与数组的顺序无关，我们可以先对数组进行排序，接下来使用二分查找，找到第一个大于等于 x 以及最后一个大于等于 x 的下标。就能得到结果了。

T4 01串

给定一个长度为 n 的 01 串，请问其中有多少个 1 的连通块？

一个 1 的连通块是指一个区间 $[l, r]$ 同时满足以下三个条件：

- $\forall i \in [l, r], s_i = 1$
- $l = 1$ 或 $s_{l-1} = 0$
- $r = n$ 或 $s_{r+1} = 0$

推荐时间复杂度 $O(n)$ 。

tag

性质观察

solve

固然这题可以直接遍历求出，但是这里介绍一个关于 1 的连通块的重要性质：

我们观察一些 1 的连通块，看看他们有什么共同点：

- 0011111000
- 01110
- 010

你发现了什么吗？

这些串总是以 01 开头，并且以 10 结尾。但是形如 111 这样的串却又不满足这样的规则，但是，我们可以在他们的头和尾各加上一个 0，变成 01110，这样又符合我们观察到的规则。

基于这样的观察，可以得出：一个 1 的连通块总是对应着恰好一个 01 和 恰好一个 10。因此，最终的 1 的连通块数量就是 01 的数量 / 10 的数量。（记得给头尾加上 0 噢）

课后练习

[C - Black Intervals](#)

T5 接雨水

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。
([链接](#))

推荐时间复杂度 $O(n)$ 。

tag

前缀后缀

solve

这题有各种各样的方法，这里介绍一个最直观的。

首先我们要注意到，一个格子最多能装多少水，取决于 $\min\{\text{左侧最高的柱子}, \text{右侧最高柱子}\}$ 。如果该值小于等于当前格子高度，代表不能接水；否则，这个柱子就能装 该值 - 当前柱子高度 的水。

于是，我们建立两个数组 pre_i 和 suf_i ，用于存储 $1 \sim i$ 的最大值和 $i \sim n$ 的最大值。最终，第 i 个柱子能接水的大小就是 $\max\{0, \min\{pre_{i-1}, suf_{i+1}\} - h_i\}$ 。

T6 只删一个

给定一个长度为 n 的数组 a ，你必须恰好删除一个数，求操作之后整个数组的 gcd 可能的**最大值**。

推荐时间复杂度 $O(n)$ 。（忽略计算 gcd 产生的复杂度）

tag

前缀后缀，数学

solve

首先，如果直接枚举删除的数，再一次求出整个数组的 gcd 最大值，复杂度为 $O(n^2)$ ，不可接受。

我们发现，在删除第 i 个数的时候，最后整体的 gcd 可以由 $1 \sim i - 1$ 的 gcd 与 $i + 1 \sim n$ 的 gcd 快速获得。因此，我们可以建立两个数组 pre_i 和 suf_i ，用于计算 $1 \sim i$ 的 gcd 和 $i \sim n$ 的 gcd。显然，这两个数组都可以 $O(n)$ 递推出来。最后，枚举每一个 i ，计算 $\gcd(pre_{i-1}, suf_{i+1})$ 并取最大值作为答案。

这种维护前后缀数组的技巧非常常用，对于恰好删除一个数或者恰好删除一个区间的问题经常可以使用。

课后练习

[Problem - D - Codeforces](#)

T7 最大子段和

给定一个长度为 n 的数组 a ，请你求出所有子区间和的**最大值**。(链接)

推荐时间复杂度 $O(n)$ 。

tag

前缀和

solve

关于区间和问题，我们首先考虑前缀和。记 $sum_i = a_1 + a_2 + \cdots + a_i$ ，那么，原题转化成最大化 $sum_r - sum_{l-1} (l \leq r)$ 。

如果枚举每一个 r ，此时 sum_r 的值固定，那么我们只需要最小化 sum_{l-1} 即可。可以发现， $l-1$ 的范围就是 $[1, r-1]$ ，也就是说，我们只需要维护当前位置前面的前缀和最小值，就可以得到当前位置可能的最大区间和了。

课后练习

[U606758 超级区间和（暂无数据） - 洛谷](#)

T8 区间和恰好为 k

给定一个长度为 n 的数组 a ，请你求出有多少子区间和恰好为 k 。 [\(链接\)](#)

推荐时间复杂度 $O(n \log n)$ 。

tag

前缀和，map

solve

类似 T7 的想法，我们记 $sum_i = a_1 + a_2 + \cdots + a_i$ ，那么，原题转化成有多少 $1 \leq l \leq r \leq n$ 满足 $sum_r - sum_{l-1} = k (l \leq r)$ 。

继续考虑枚举 r ，此时 sum_r 固定，对原式变形得到 $sum_{l-1} = sum_r - k$ 。也就是，对于每一个 r ，查找在这之前有多少个前缀和等于 $sum_r - k$ ，这里计数我们可以使用 STL 里的 map 容器。

课后练习

[cf2121F](#)

T9 最长不相交区间

给定 n 个区间 $[l_i, r_i]$ ，从中选出一些区间，使这些区间互不相交。请问最多能选出多少个？ [\(链接\)](#)

推荐时间复杂度 $O(n \log n)$ 。

tag

排序，贪心

solve

可以证明，一定有至少一个最优解包含最早结束的区间。于是，我们先按照右端点 r 对区间排序，接下来顺着遍历模拟，如果有一个区间的左端点大于当前右端点了，我们就将这个区间加入答案，以此类推向后更新。

定理正确性的证明可以使用反证法：（以下结束最早的区间指右端点 r 最小的区间）

- 假设所有最优解都不包含最早结束的区间（不妨记为 $[a_0, b_0]$ ），记其中一个解为 $[a_1, b_1], [a_2, b_2], \dots$
- 因为答案不包含最早结束的区间，所以 $b_0 < b_1$ 。并且该解之间各个区间不相交，所以有 $b_1 < a_2$ 。

- 因此我们得到 $b_0 < b_1 < a_2$, 即 $b_0 < a_2$ 。于是, 我们可以把 $[a_1, b_1]$ 换成 $[a_0, b_0]$ 。此时仍然有各个区间不相交, 并且长度不变, 所以也是一个最优解。
- 与原假设矛盾, 因此一定有至少一个最优解包含最早结束的区间。

现在我们证明了, 第一个要选的一定是最早结束的区间。那么第二个区间该怎么选择? 也就是说, 我们在所有不与第一个选择的区间相交的区间中, 选出最多的不相交区间。我们发现, 这是一个与原问题一样的子问题, 于是我们也应当在剩下的区间中选择最早结束的区间。

T10 安排房间

给定 n 个区间 $[l_i, r_i]$, 请问被覆盖最多次数的点被覆盖了几次? ($1 \leq l_i \leq r_i \leq 10^9$) (没有链接)

推荐时间复杂度 $O(n \log n)$ 。

tag

离散化, 差分

solve

可以看出, 这其实是一个经典的差分问题。我们记数组 $diff_i = a_i - a_{i-1}$, 那么, 我们给 $diff_i$ 上加 k , 再对 $diff$ 做一个前缀和, 相当于在 a_1 到 a_k 都加上了 k ; 如果我们给 $diff_r$ 上加 k , 再给 $diff_{l-1}$ 减去 k , 再做前缀和, 就相当于在 a 数组上给下标 $[l, r]$ 都加上 k 。

通过差分, 我们可以在 $O(1)$ 的时间内完成区间加操作 (但是必须要花费 $O(n)$ 的时间做一遍前缀和才能得到结果)。然鹅, 题里的区间高达 10^9 , 显然开一个这么大的数组是不现实的, 于是我们考虑离散化。

何为离散化, 就是把原题范围从与值有关变到与数量有关。固然原题的范围是 $[1, 10^9]$, 但实际上, 真正有关的值只有 n 个, 其他的值都是无关的。因此, 我们只把这些有关的值拿出来, 按照大小顺序给他编上号, 然后再进行差分操作就好了。

具体实现可以看 code, 或者自行学习离散化。

T11 铺设道路

给定一个长度为 n 的序列 a , 你可以进行任意次如下操作:

- 选择两个整数 l, r , 使得任意 $l \leq i \leq r$ 有 $a_i \neq 0$;
- 将所有 $l \leq i \leq r$ 的 a_i 减去一。

求出最小操作次数, 让 a 全为 0。 ([链接](#))

推荐时间复杂度 $O(n)$ 。

tag

贪心

solve

我们贪心的思考, 每次减去的数目想要最多, 就要尽量让值大的数带着小的数一起剪掉。

不妨从左到右考虑：如果前一个的值大，那么当前值就可以在前一个值减的时候顺带减少；否则，如果当前值更大，就必须多花费 $a_i - a_{i-1}$ 的操作次数来减。于是我们得到答案：

$$ans = \sum_{i=2}^n \max\{0, a_i - a_{i-1}\}.$$

T12 绝对值之和

给定一个长度为 n 的序列 a ，求出函数 $f(x) = \sum_{i=1}^n |x - a_i|$ 的最小值，并输出一个能取到最小值的 x_0 。

推荐时间复杂度 $O(n \log n)$ 。

tag

数学

solve

不妨先对 a 进行升序排序，这样最小值为 a_1 ，最大值为 a_n 。显然有 $x_0 \in [a_1, a_n]$ ，在这种条件下，我们有 $|a_1 - x_0| + |a_n - x_0| = a_n - a_1$ ，我们发现这玩意算出来居然是个定值，也就是说，接下来我们只需要最小化 $\sum_{i=2}^{n-1} |x - a_i|$ ，类似的，显然有 $x_0 \in [a_2, a_{n-1}] \dots$

如此递归下去，最终我们按照 n 的奇偶性进行讨论：

- 当 n 为奇数的时候，最终 x_0 只能取值为 a 的中位数 $a_{(n+1)/2}$ ；
- 当 n 为偶数的时候，最终 x_0 可以取值到区间 $[a_{n/2}, a_{n/2+1}]$ 。

最终最小值可以通过 x_0 代回 $f(x)$ 求出。

课后练习

你能在 $O(n \log n)$ 的时间复杂度求出所有 $f(a_i)$ 吗？

T13 区间异或和

给定 q 次询问，每次询问查询区间 $[l, r]$ 的异或和，即求 $\bigoplus_{i=l}^r i$ 。 ($0 \leq l \leq r \leq 10^{18}$)

推荐时间复杂度 $O(q)$ 。

tag

位运算，打表

solve

通过数据范围，我们发现这是一个需要 $O(1)$ 出结果的题，于是我们考虑打表。

然而直接打表也不是很好写，我们回忆起异或有这样的性质： $a \oplus a = 0$ 。不妨记 $f(x) = \bigoplus_{i=1}^x i$ ，于是我们有： $\bigoplus_{i=l}^r i = f(l-1) \oplus f(r)$ 。

接下来，我们使用打表大法，暴力求出 $f(1), f(2), f(3), \dots$ ，我们发现一个惊人的规律！

$$f(n) = \begin{cases} n & n \equiv 0 \pmod{4} \\ 1 & n \equiv 1 \pmod{4} \\ n+1 & n \equiv 2 \pmod{4} \\ 0 & n \equiv 3 \pmod{4} \end{cases}$$

这样，我们就能愉快的求出答案了！

同样打表可知 $\bigoplus_{i=1}^n \bigoplus_{j=1}^i j$ 有类似的周期性，且周期为 8。

课后练习

[J-小王的异或问题 2025年第一届上海师范大学程序设计竞赛（同步赛）](#)

MID

T1 分数规划 I

有 n 个物品，每个物品有两个权值 a_i, b_i ，先从中选出恰好 k 个物品，求出 $\frac{\sum a_i}{\sum b_i}$ 的最大值/最小值。

推荐时间复杂度 $O(n \log^2 n)$ 。

tag

二分答案，贪心

solve

不妨先考虑最大值。

直接求出最大值过于困难，于是我们考虑二分一个最大值，使原问题变成可行性问题。我们考虑答案 x ，也就是问是否存在一个选择方式，使 $\frac{\sum a_i}{\sum b_i} \geq x$ 。

进行如下变形：

$$\begin{aligned}\sum a_i - x \sum b_i &\geq 0 \\ \Leftrightarrow \sum (a_i - x b_i) &\geq 0\end{aligned}$$

对于一个答案，我们要在恰好选 k 物品的条件下最大化 $\sum (a_i - x b_i)$ 。我们可以直接将 $a_i - x b_i$ 按大小排序，然后选出前 k 大的加起来，判断是否 ≥ 0 就行了。

最小值也是类似的想法，在这里不再赘述。

T2 分数规划 II

有 n 个物品，每个物品有两个权值 a_i, b_i ，先从中选出若干个物品，在 $\sum b_i \geq W$ 的前提下，求出 $\frac{\sum a_i}{\sum b_i}$ 的最大值。[题目链接](#)

推荐时间复杂度 $O(nW \log n)$ 。

tag

二分答案，背包

solve

有了分数规划 I 的做题经验，我们可以继续使用二分答案的算法，但是在考虑可行性时，我们既要最大化 $\sum (a_i - x b_i)$ ，又要保证 $\sum b_i \geq W$ 。这不禁让我们想起来背包 DP。

我们将 b_i 看作重量， $a_i - x b_i$ 看作价值，做背包 DP，并且 $\sum b_i \geq W$ 时一律视作 W 。这样， $dp[n][W]$ 就是我们要的满足条件的最大值。

T3 分数规划 III

给定一个长度为 n 的数组 a ，在所有长度 $\geq L$ 的子数组中，找出子数组的最大平均值。 [\(链接\)](#)

推荐时间复杂度 $O(n \log n)$ 。

tag

二分答案，前缀和

solve

经过了前两题的洗礼，很容易看出这题就是一个二分答案。并且他还涉及到一个子区间和问题，我们也先顺便求个前缀和。

考虑答案 x ， $\frac{sum_r - sum_{l-1}}{r - l + 1} \geq x \Leftrightarrow (sum_r - sum_{l-1}) - x(r - l + 1) \geq 0$ 。这种形式让我们回想起 EZ 中的另一个 trick：考虑枚举右端点 r ，继续将上式变形：

$$\begin{aligned}(sum_r - sum_{l-1}) - x(r - l + 1) &\geq 0 \\ \Leftrightarrow (sum_r - xr) - (sum_{l-1} - x(l - 1)) &\geq 0\end{aligned}$$

此时 $sum_r - xr$ 是定值，我们只需要维护 $r - l + 1 \geq L$ 的情况下， $sum_{l-1} - x(l - 1)$ 的最小值就好啦。

T4 菱形前缀和

给定一个 $n \times m$ 的二维数组 a ，接下来共进行 q 次操作，第 i 次操作给所有满足 $|x - a_i| + |y - b_i| \leq r_i$ 的 $a_{x,y}$ 增加 1（可以发现这是一个菱形区域）。请输出所有操作之后的 a 数组。

推荐时间复杂度 $O(nm)$ 。

tag

前缀和，数学

solve

前置知识：曼哈顿距离与切比雪夫距离

- 两个点的曼哈顿距离： $dis = |x_1 - x_2| + |y_1 - y_2|$
- 两个点的切比雪夫距离： $dis = \max\{|x_1 - x_2|, |y_1 - y_2|\}$
- 不难看出，与某点 P 的曼哈顿距离 $\leq k$ 的点是一个菱形；而与某点 P 的切比雪夫距离 $\leq k$ 的点是一个正方形。

由于曼哈顿距离是菱形的，如果用它去做前缀和非常麻烦；但是在切比雪夫距离下，做前缀和很简单（就是我们学过的二位前缀和），于是我们想，能不能用某种方法把曼哈顿距离转化为切比雪夫距离呢？

通过数学变形，我们有一下事实：

- 将点 (x, y) 的坐标变为 $(x + y, x - y)$ ，原坐标系中两点的曼哈顿距离 = 新坐标系的切比雪夫距离；
- 将点 (x, y) 的坐标变为 $(\frac{x + y}{2}, \frac{x - y}{2})$ ，原坐标系中两点的切比雪夫距离 = 新坐标系的曼哈顿距离；

这样，在进行题目中的操作前，我们先将几个关键点转化成 $(x + y, x - y)$ 的坐标，按照二位前缀和的形式操作，然后做一遍前缀和，最后把所有点转化成 $(\frac{x + y}{2}, \frac{x - y}{2})$ 的形式输出就好了。

需要注意的是，转化后的坐标往往会带有负数，这对代码细节处理和调试能力要求较高，所以谨慎使用该方法。

课后练习

[T592948 Cookie Clicks Buttons - 洛谷](#)

T5 摧毁路线

给定一个 n 个节点， m 条边的简单无向图。接下来，对这张图进行 q 次操作：

1. 删除这张图中的编号为 x 的边；
2. 询问是否存在一条路径使得 y 号节点能到达 z 号节点。

对于每个操作 2，输出 YES 或者 NO。

推荐时间复杂度 $O(m \log m + q)$ 。

tag

并查集，离线处理

solve

在大部分情况下，删边都是非常头痛的操作，我们似乎没有什么简单的数据结构可以有效维护删边操作；但是它的反向操作——即加边，我们可以用并查集轻松地维护。

基于这样的思考，我们考虑先把所有操作都存起来，然后再逆序一个一个处理。也就是说：先建出所有操作中没有被删的边，然后逆序处理操作，如果是操作 1，则用并查集加边，如果是操作 2，就用并查集查询连通性。

遇到删边就离线处理是一个常用 trick，但是也有自身的局限性，遇到类似的题目时一定要多加思考。

T6 代价转移

给定正整数 a 和 b ，你每次可以从下面三种操作中选择任意一种：

- 将 a 加一： $a \leftarrow a + 1$ ，代价为 C_1 ；
- 将 a 减一： $a \leftarrow a - 1$ ，代价为 C_2 ；
- 将 a 乘二： $a \leftarrow a \times 2$ ，代价为 C_3 。

计算将 a 变成 b 的最小代价。[\(链接\)](#)

推荐时间复杂度 $O(\max(a, b) \log \max(a, b))$ 。

tag

图论建模，最短路

solve

一道经典的图论建模题。我们看到题目中的求最小代价，不禁想到求最短路。由于题目中的值域很小，我们考虑让每个数字代表一个图中的点，这样，每一种转移方式都是图上的一条边；而转移的代价，就是我们边上权值。于是原问题就变成了求 a 到 b 的最短路。

需要注意的是，本题应该有 $2 \max(a, b)$ 个点，这是由于一个点可以先乘以 2 再一点点减到目标点。至于更大的点，可以证明都是用不上的。

课后习题

[Problem - 436C - Codeforces](#)

T7 最大正方形

给定一个 $n \times n$ 的二维网格 a ，且 $a_{ij} \in \{0, 1\}$ 。对于每个格子 (i, j) ，求出满足以下条件的最大 r 。

- 对于任意 (x, y) 且 $\max(|x - i|, |y - j|) \leq r$ ，有 $a_{xy} = 0$ 。

即在所有以 (i, j) 为中心且正方形内全为数字 0 的正方形中，找到边长最大的那一个。

推荐时间复杂度 $O(n^2)$ 。

tag

BFS

solve

先给出一个不太优秀的 $O(n^2 \log n)$ 做法供参考。

对整个网格做二维前缀和。对于每一个点，我们不妨二分它的 r ，在检查可行性时，如果整个正方形的前缀和是 0，则可行；否则不可行。

关键观察：以一个点为中心的正方形，它的最大边长就是该点到的最近的 1 的切比雪夫距离。

我们发现，直接求出每个点最近的 1 的距离太过麻烦，那么有没有什么办法可以求出 1 到我们的最近距离？有的兄弟，有的。

我们对图中所有的 1 进行多源 BFS，并且每次扩展周围 8 个格子，由于距离的双向性，所有 1 到一个格子的最短距离就是这个格子离最近的 1 的距离。

并且，一次往周围扩展 8 个格子，保证了每次扩展都使切比雪夫距离增加 1，这样就符合我们的观察了。

最后输出这个最短路数组就可。

T8 最大菱形

给定一个 $n \times n$ 的二维网格 a ，且 $a_{ij} \in \{0, 1\}$ 。对于每个格子 (i, j) ，求出满足以下条件的最大 r 。

- 对于任意 (x, y) 且 $|x - i| + |y - j| \leq r$ ，有 $a_{xy} = 0$ 。

即在所有以 (i, j) 为中心且菱形内全为数字 0 的菱形中，找到边长最大的那一个。

推荐时间复杂度 $O(n^2)$ 。

tag

BFS

solve

通过我们学过的曼哈顿转切比雪夫的 trick (T4)，我们可以仿照 T7 的不太优秀做法给出一个同样不太优秀的 $O(n^2 \log n)$ 做法。

但是我们也可以类似的进行多源 BFS。需要注意的是，这次我们一次只能扩展上下左右四个格子，这是由于这里给出的是[曼哈顿距离](#)，一次扩展 4 个恰好使曼哈顿距离 +1。

课后练习

[C-总辖之愿 金山杯2025年武汉理工大学程序设计竞赛](#) (笑点解析：主包赛场上调了 3h 勉强调出来那个“不太优秀的做法”，最后一看题解天塌了)

T9 位运算

给定一个长度为 n 的序列 a ，共有 q 次询问，每次询问给出 l, r ，求出最大的 $r \leq n$ ，使得 $a_l \wedge a_{l+1} \wedge \dots \wedge a_r \leq k$ 。 [\(链接\)](#)

推荐时间复杂度 $O(n \log^2 n)$ 。

tag

二分答案，位运算，st 表

solve

这题的最优解是时间复杂度 $O(n \log n)$ 的 st 表，但是这里给出一个更加经典的 trick 做法。

首先需要注意到，按位与 $\wedge$ 总是单调不增的，也就是你与上的数越多，你的值只会更小；而按位或 $\vee$ 则相反，它是单调不减的；按位异或 $\oplus$ 没有单调性。

通过单调性，我们确定了第一步是二分答案：二分一个右端点 r ，检查是否满足题目条件。

接下来的步骤是 trick 的关键。由于按位与没有加法很多优美的性质，所以不能通过前缀和得到答案。但是，**各种位运算对每一位的操作都是独立的**，每一位之间相互不影响，于是我们可以对数的每一个二进制位分开去考虑。

我们考虑每一个二进制位的结果，并整合成答案：对于第 i 个二进制位， $\wedge$ 完之后变成 1 当且仅当整个 $[l, r]$ 范围内的 a_k ，在这一位都是 1。如何判断整个区间是否都是 1？答案是前缀和，如果当前区间和 = 区间长度，结果的这一位就是 1，否则就是 0。

课后练习

[abc408 - E - Minimum OR Path](#)

[Problem - 1624G - Codeforces](#)

T10 中位数

给定一个长度为 n 的序列 a ，对于每一个奇数 i ，求出序列中前 i 项的中位数。 [\(链接\)](#)

tag

树状数组，堆

solve

这题本质上就是实现一个数据结构来维护整个数组的中位数，并支持插入操作。这里给出两种做法：

对顶堆做法

我们知道，堆可以维护一个序列的最大值，也可以维护一个序列的最小值，那么有没有什么办法可以维护中位数呢？答案是可以的，这就是对顶堆。

我们考虑两个堆，一个大根堆，一个小根堆：我们要保证小根堆的堆顶大于大根堆的堆顶，由于小根堆的堆顶是最小值，那么这就保证了小根堆的所有值都大于大根堆的值。这样，如果小根堆内有 k 个元素，那么前 k 大的元素都在小根堆中，我们就可以通过保证小根堆的元素不变来维护第 k 大的元素了。

那么类似的，通过这种方法来保证小根堆的元素数量为 $\lfloor \frac{n}{2} \rfloor$ ，就能维护出中位数了。

实现的时候，每次插入都要对比一下小根堆的堆顶：如果比堆顶大，就把原堆顶扔到大根堆里去，再插入到小根堆里；否则直接扔到大根堆里。然后再根据堆的大小处理。

树状数组/线段树做法

首先由于值域比较大，我们需要将这些数离散化下来，不妨设离散化后第 i 个数为 d_i 。每次我们插入一个数字，实际上就是给 d_i 这个位置 $+1$ 。

接下来考虑如何求出中位数。我们知道如果 x 是中位数，那么小于等于它的数应该至少有 $\lfloor \frac{n}{2} \rfloor + 1$ 个，也就是说，区间 $[1, x]$ 的和应当 $\geq \lfloor \frac{n}{2} \rfloor + 1$ 。显然， x 具有单调性，因此我们可以通过二分+线段树/树状数组或者线段树/树状数组上二分来求出中位数。

HARD

T1 区间 LCA

给定一颗节点数为 n 、根为 1 的树。接下来有 q 次查询：

- $l\ r$ 代表询问编号 l 到编号 r 的所有节点的 LCA。

推荐时间复杂度 $O(n \log n + q \log n)$ 。

tag

LCA, dfn 序

solve

可以证明，编号 l 到编号 r 的所有节点的 LCA 就是这些节点中，dfn 序最小的节点与 dfn 序最大的节点的 LCA。

具体证明可以参考 DFS 序求 LCA 的原理，比如下面这篇文章：[冷门科技 —— DFS 序求 LCA - 洛谷专栏](#)

UNK

括号序列，01序列，+1-1序列转换：https://www.luogu.com.cn/problem/AT_agc056_c

线段树维护矩阵乘法

01原理

图转树：CF901D

感觉群u都写过吧，1039的E1，就是问你一个数组是否有至少长度为k的子数组满足中位数为x，做法就是二分答案然后维护一个数组将大于等于二分的那个数标记为1，反之-1，然后做前缀和，如果有长度至少为k的子数组且区间和大于等于0说明这个x可以作为子数组的中位数

gcd popcount什么的，都是log次之后就会停止变化

图与矩阵乘法：<https://www.luogu.com.cn/problem/P5789>

反图：反向跑最短路，方便判断正向的可达性，做到碰上思路提前剪枝，顺便可以延伸出来，正反各跑一次最短路，就可以做到O1求出来，必须走某条边的前提下的最短路，<https://www.luogu.com.cn/problem/solution/CF645D>

把数组分成连续的符合要求的几块，求最大数量。